CSS 262: Linux Administration

Package Management & Repositories

Lecture 7: Software Packages, Repos & Building from Source

📋 Today's Agenda

Part 1: Package Management Fundamentals

- Why package managers?
- Package formats: .deb vs .rpm
- Package lifecycle

Part 2: APT Deep Dive (Debian/Ubuntu)

- apt commands & workflows
- dpkg low-level operations
- Pinning & holding packages

Part 3: DNF Deep Dive (Fedora/RHEL)

- dnf commands & workflows
- rpm low-level operations
- Groups & modules

Part 4: Repos & Building from Source

- Repository management & GPG keys
- Adding third-party repos
- Compiling from source

🎯 **Learning Objective:** Install, manage, and troubleshoot software packages using APT/DNF, configure repositories, and build software from source.

🔄 Quick Recap: Week 6

Linux Networking Basics

- **TCP/IP Stack**: Layers, ports, IP addressing
- **Interfaces**: `ip link`, `ip addr`, naming conventions
- **Configuration**: nmcli, Netplan, static IPs
- **DNS**: resolv.conf, `dig`, `nslookup`
- **Troubleshooting**: ping, traceroute, `ss -tuln`

✅ You should now be able to configure network interfaces and troubleshoot connectivity!

Part 1: Package Management Fundamentals

Why Package Managers Matter

📦 Why Package Managers?

Without a Package Manager

- Manually download tarballs
- Resolve dependencies yourself
- No automatic security updates
- No easy uninstall/rollback
- Version conflicts everywhere

With a Package Manager

- One-command install/remove
- Automatic dependency resolution
- Security patches via updates
- Verified & signed packages
- Clean uninstallation

Three Key Benefits

1. Dependency Management

```
nginx → libssl → libcrypto
      → libpcre → libc
      → zlib
```

2. Versioning & Updates

```
apt upgrade   # Patch everything
```

3. Security

- GPG-signed packages
- Verified checksums
- Trusted repo sources

🔀 Package Manager Ecosystem

Two Major Families

Feature	Debian/Ubuntu	Fedora/RHEL
Package format	`.deb`	`.rpm`
Low-level tool	`dpkg`	`rpm`
High-level tool	`apt`	`dnf` (yum legacy)
Repo config	`/etc/apt/sources.list`	`/etc/yum.repos.d/`
Third-party	PPA	COPR

Package Contents

A package bundles: **binaries**, **config files**, **metadata** (name, version, deps), and **scripts** (pre/post install/remove hooks).

.deb vs .rpm Internals

Debian Package (.deb)

```
# Inspect .deb contents
dpkg-deb --info package.deb
dpkg-deb --contents package.deb

# Extract without installing
dpkg-deb -x package.deb ./output/
```

Structure:

```
package.deb
├── debian-binary    (format version)
├── control.tar.gz   (metadata)
│   ├── control      (name, deps)
│   ├── preinst      (pre-install)
│   └── postinst     (post-install)
└── data.tar.gz      (actual files)
```

RPM Package (.rpm)

```
# Inspect .rpm contents
rpm -qpi package.rpm
rpm -qpl package.rpm

# Extract without installing
rpm2cpio package.rpm | cpio -idmv
```

Structure:

```
package.rpm
├── Lead       (magic number)
├── Signature  (GPG/MD5)
├── Header     (metadata, deps)
└── Payload    (cpio archive)
```

🔄 Package Lifecycle

Common Operations Across Both Families

Action	APT (Debian/Ubuntu)	DNF (Fedora/RHEL)
Update index	`apt update`	`dnf check-update`
Install	`apt install nginx`	`dnf install nginx`
Remove	`apt remove nginx`	`dnf remove nginx`
Purge config	`apt purge nginx`	`dnf remove nginx` (configs stay)
Upgrade all	`apt upgrade`	`dnf upgrade`
Search	`apt search nginx`	`dnf search nginx`
Info	`apt show nginx`	`dnf info nginx`
List installed	`apt list --installed`	`dnf list installed`

Part 2: APT Deep Dive

Debian & Ubuntu Package Management

🐧 APT Essentials

Update & Upgrade

```
sudo apt update              # Refresh package index from repos
sudo apt upgrade             # Upgrade all installed packages
sudo apt full-upgrade        # Upgrade + remove obsolete packages
sudo apt dist-upgrade        # Handle changing dependencies
```

Install & Remove

```
sudo apt install nginx       # Install a package
sudo apt install nginx=1.18.0-0ubuntu1  # Install specific version
sudo apt install -y nginx curl git      # Install multiple, skip prompt
sudo apt remove nginx        # Remove package (keep config)
sudo apt purge nginx         # Remove package + config files
sudo apt autoremove          # Remove unused dependencies
```

🔍 APT: Search & Inspect

Searching Packages

```
apt search nginx            # Search by name/description
apt list --installed        # List all installed packages
apt list --upgradable       # List packages with updates
```

Package Details

```
apt show nginx              # Detailed package info
apt-cache depends nginx     # Show dependencies
apt-cache rdepends nginx    # Show reverse dependencies (who needs this?)
apt-cache policy nginx      # Show available versions & priorities
```

🔧 dpkg: Low-Level Operations

Direct Package Management

```
sudo dpkg -i package.deb       # Install a .deb file
sudo dpkg -r package-name      # Remove package
sudo dpkg -P package-name      # Purge (remove + config)
```

Querying Installed Packages

```
dpkg -l                        # List all installed packages
dpkg -l | grep nginx           # Filter for specific package
dpkg -L nginx                  # List files installed by package
dpkg -S /usr/sbin/nginx        # Which package owns this file?
dpkg --get-selections          # Export installed package list
```

Fixing Broken Installs

```
sudo dpkg --configure -a       # Configure pending packages
sudo apt --fix-broken install  # Resolve dependency issues
```

💡 **Tip:** Use `apt` for daily tasks; use `dpkg` when installing standalone `.deb` files or troubleshooting.

📌 APT: Pinning & Holding Packages

Hold a Package (Prevent Upgrades)

```
sudo apt-mark hold nginx          # Prevent nginx from upgrading
sudo apt-mark unhold nginx        # Allow upgrades again
apt-mark showhold                 # List held packages
```

APT Pinning (Advanced)

```
# /etc/apt/preferences.d/nginx
Package: nginx
Pin: version 1.18.*
Pin-Priority: 1001
```

Priority	Behavior
1001+	Force install even if downgrade
500	Normal (default)
-1	Never install

Part 3: DNF Deep Dive

Fedora & RHEL Package Management

🎩 DNF Essentials

Update & Upgrade

```
sudo dnf check-update           # Check for available updates
sudo dnf upgrade                 # Upgrade all packages
sudo dnf upgrade --security      # Only security updates
sudo dnf distro-sync            # Sync to latest in repo
```

Install & Remove

```
sudo dnf install nginx           # Install a package
sudo dnf install nginx-1.20.1    # Specific version
sudo dnf install -y nginx curl git  # Multiple, skip prompt
sudo dnf remove nginx            # Remove package
sudo dnf autoremove              # Remove unused dependencies
```

💡 **DNF vs YUM:** `dnf` replaced `yum` in Fedora 22+ and RHEL 8+. On RHEL 8+, `yum` is a symlink to `dnf`.

🔍 DNF: Search & Inspect

Searching Packages

```
dnf search nginx                # Search by name/summary
dnf search all nginx            # Search all metadata fields
dnf list installed              # List installed packages
dnf list available              # List all available packages
```

Package Details

```
dnf info nginx                  # Show package info
dnf repoquery --requires nginx  # Show dependencies
dnf repoquery --whatrequires nginx  # Reverse dependencies
dnf provides /usr/sbin/nginx    # Which package provides this file?
```

History & Rollback

```
dnf history                     # Show transaction history
dnf history info 15             # Details of transaction 15
sudo dnf history undo 15        # Undo a specific transaction
```

🔧 rpm: Low-Level Operations

Direct Package Management

```
sudo rpm -ivh package.rpm        # Install (-i install, -v verbose, -h hash)
sudo rpm -Uvh package.rpm        # Upgrade (or install if missing)
sudo rpm -e package-name         # Erase (remove) package
```

Querying Installed Packages

```
rpm -qa                          # List all installed
rpm -qa | grep nginx             # Filter for package
rpm -qi nginx                    # Info about installed package
rpm -ql nginx                    # List files from package
rpm -qf /usr/sbin/nginx          # Which package owns this file?
rpm -qR nginx                    # Show dependencies
```

Verify Package Integrity

```
rpm -V nginx                     # Verify installed package files
rpm --checksig package.rpm       # Verify GPG signature
```

📚 DNF Groups & Modules

Package Groups

```
dnf group list                    # List available groups
dnf group info "Development Tools"  # Show group contents
sudo dnf group install "Development Tools"  # Install entire group
sudo dnf group remove "Development Tools"   # Remove group
```

Common Groups

Group	Includes
Development Tools	gcc, make, autoconf, git
System Tools	lsof, strace, tcpdump

Modules (RHEL 8+)

```
dnf module list                   # List available modules
sudo dnf module enable nodejs:18  # Enable Node.js 18 stream
sudo dnf module install nodejs:18 # Install from stream
```

Part 4: Repository Management

Sources, GPG Keys & Security

🗄️ What Are Repositories?

APT Repositories (Debian/Ubuntu)

```
# /etc/apt/sources.list
deb http://archive.ubuntu.com/ubuntu \
  focal main restricted universe multiverse

deb http://archive.ubuntu.com/ubuntu \
  focal-updates main restricted
```

Components:

- `main` – free, supported
- `restricted` – proprietary drivers
- `universe` – community-maintained
- `multiverse` – non-free

DNF Repositories (Fedora/RHEL)

```
# /etc/yum.repos.d/fedora.repo
[fedora]
name=Fedora $releasever - $basearch
baseurl=https://download.fedoraproject.org/pub/
  fedora/linux/releases/$releasever/Everything/
  $basearch/os/
enabled=1
gpgcheck=1
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-fedora
```

Fields:

- `enabled` – 0 or 1
- `gpgcheck` – verify signatures
- `gpgkey` – signing key path

➕ Adding Third-Party Repos

PPA on Ubuntu

```
# Add a PPA
sudo add-apt-repository \
  ppa:ondrej/nginx
sudo apt update
sudo apt install nginx

# Remove a PPA
sudo add-apt-repository --remove \
  ppa:ondrej/nginx
```

Manual APT Repo

```
# Add repo + key (modern method)
curl -fsSL https://example.com/key.gpg \
  | sudo gpg --dearmor \
  -o /usr/share/keyrings/example.gpg

echo "deb [signed-by=...example.gpg] \
  https://repo.example.com stable main" \
  | sudo tee /etc/apt/sources.list.d/example.list
```

COPR on Fedora

```
# Enable a COPR repository
sudo dnf copr enable user/project
sudo dnf install package-name

# Disable
sudo dnf copr disable user/project
```

Manual DNF Repo

```
# /etc/yum.repos.d/example.repo
[example]
name=Example Repository
baseurl=https://repo.example.com/el/$releasever/
enabled=1
gpgcheck=1
gpgkey=https://repo.example.com/RPM-GPG-KEY
```

🔑 GPG Key Verification

Why GPG Keys?

- Ensures packages come from **trusted sources**
- Prevents tampering (MITM attacks)
- Every official repo ships a public GPG key

APT Key Management

```
# Modern approach (preferred)
curl -fsSL https://example.com/key.gpg \
  | sudo gpg --dearmor -o /usr/share/keyrings/example.gpg

# Legacy approach (deprecated)
curl -fsSL https://example.com/key.gpg | sudo apt-key add -
apt-key list                      # List trusted keys
```

RPM Key Management

```
sudo rpm --import https://example.com/RPM-GPG-KEY
rpm -qa gpg-pubkey*               # List imported keys
rpm -qi gpg-pubkey-XXXXXXXX       # Key details
```

⚠️ Security Implications of Third-Party Repos

Risks

Risk	Description
Malicious packages	Repo owner can push backdoored binaries
Dependency hijacking	Overrides official packages with higher version
Abandoned repos	No security updates, stale packages
Key compromise	Attacker signs malicious updates

Best Practices

1. **Audit the source** – Only add repos from trusted vendors
2. **Pin priorities** – Prevent third-party repos from overriding system packages
3. **Use GPG verification** – Never skip signature checks
4. **Minimize repos** – Fewer repos = smaller attack surface
5. **Review before updating** – `apt upgrade --dry-run` / `dnf upgrade --assumeno`

Part 5: Building from Source

When Packages Aren’t Enough

🔨 When to Build from Source?

Reasons

- **No package available** for your distro
- Need **custom compile flags** (e.g., enable specific modules)
- Require a **newer version** than the repo provides
- **Patching** source code for specific fixes
- **Learning** how software is built

Risks

- No automatic security updates
- Manual dependency tracking
- Potential file conflicts with packaged versions
- Harder to uninstall cleanly

⚠️ **Rule of thumb:** Always prefer a package if one exists. Build from source only when necessary.

🛠️ The Classic Build Flow

```
# 1. Install build tools
sudo apt install build-essential              # Debian/Ubuntu
sudo dnf group install "Development Tools"    # Fedora/RHEL

# 2. Download & extract source
wget https://example.com/software-1.2.3.tar.gz
tar xzf software-1.2.3.tar.gz && cd software-1.2.3/

# 3. Configure → Compile → Install
./configure --prefix=/usr/local --enable-ssl
make -j$(nproc)
sudo make install
```

Step	Purpose
`./configure`	Detects system capabilities, sets paths & options
`make`	Compiles source code into binaries
`make install`	Copies binaries, libs, man pages to system

📦 checkinstall: Track Source Installs

Problem with `make install`

- Files scattered across `/usr/local/`
- No package manager tracking
- No clean uninstall

Solution: checkinstall

```
sudo apt install checkinstall     # Debian/Ubuntu

# Instead of 'make install', run:
sudo checkinstall --pkgname=myapp --pkgversion=1.2.3 \
  --default make install
```

This creates a `.deb` (or `.rpm`) and installs it – the package manager now tracks it!

Uninstall Cleanly

```
sudo apt remove myapp             # Clean removal via dpkg
dpkg -L myapp                     # Verify files are gone
```

📋 Build Dependencies

Install Build Deps for Existing Packages

```
sudo apt build-dep nginx          # Debian/Ubuntu
sudo dnf builddep nginx           # Fedora/RHEL
```

Common Build Dependencies

Debian/Ubuntu	Fedora/RHEL	Purpose
`build-essential`	`gcc`, `make`	C compiler & build tools
`libssl-dev`	`openssl-devel`	SSL/TLS headers
`libpcre3-dev`	`pcre-devel`	Regex library headers
`zlib1g-dev`	`zlib-devel`	Compression headers

Missing header like `openssl/ssl.h: No such file or directory`? Install the `-dev` / `-devel` package for that library.

📊 Summary: Package Management

Key Concepts Covered

1. **Package Managers** – Dependency resolution, security, lifecycle
2. **APT (Debian/Ubuntu)** – apt, dpkg, pinning, holding
3. **DNF (Fedora/RHEL)** – dnf, rpm, groups, modules
4. **Repositories** – sources.list, yum.repos.d, GPG keys
5. **Third-party Repos** – PPAs, COPRs, security risks
6. **Building from Source** – configure, make, checkinstall

Quick Reference

Task	APT	DNF
Refresh index	`apt update`	`dnf check-update`
Install	`apt install`	`dnf install`
Remove	`apt remove`	`dnf remove`
Search	`apt search`	`dnf search`
File owner	`dpkg -S`	`rpm -qf`
Add repo	`add-apt-repository`	`dnf copr enable`

🎯 Learning Objectives: Did We Achieve?

By now, you should be able to:

- ✅ Explain why package managers are essential for system administration
- ✅ Use `apt` and `dpkg` to install, remove, and query packages
- ✅ Use `dnf` and `rpm` to manage packages on Fedora/RHEL
- ✅ Search for packages and inspect dependencies
- ✅ Add and remove third-party repositories (PPA, COPR)
- ✅ Verify GPG keys and understand supply-chain risks
- ✅ Build software from source using `configure`, `make`, `checkinstall`

🎓 **Next Week:** SSH Hardening & Remote Access

🧪 Lab Practice: Package Management

Exercise 1: APT Workflow

Install `htop` and `tree` via `apt`. List files with `dpkg -L`. Remove with `--purge` and verify.

Exercise 2: Package Investigation

Map the dependency tree of `openssh-server` using `apt-cache depends/rdepends`. Find which package provides `/usr/bin/ssh`.

Exercise 3: Add a PPA

Add `ondrej/nginx` PPA, install nginx, verify newer version. Remove PPA and downgrade.

Exercise 4: Build from Source

Download `jq` source, install build deps, compile, use `checkinstall` to create a .deb.

Exercise 5: Security Audit

List all configured repos. Verify each has GPG checking enabled (`signed-by` / `gpgcheck=1`).

🔗 Additional Resources

Documentation

- APT User's Guide – Official Debian APT documentation
- DNF Command Reference – Full dnf documentation
- RPM.org – RPM packaging reference

Man Pages

- `man apt` · `man dpkg` · `man apt-cache` · `man sources.list`
- `man dnf` · `man rpm` · `man yum.conf`

Books & Guides

- *"UNIX and Linux System Administration Handbook"* – Package management chapter
- Arch Wiki: Pacman/Rosetta – Cross-distro command comparison

Questions?

Next: SSH Hardening & Remote Access